Iteration: map(), walk(), and saving many outputs

Wednesday, June 17, 2026

Today: Wednesday June 17

Before class

- **Perusall Reading**: *R4DS* Ch. 26 §26.4–26.5 - Iteration

Plan for today

- From `across()` (columns) to `map()` (any list or vector)
- The `map()` family and typed variants (`map_dbl()`, `map_chr()`, etc.)
- `walk()`
- **List-columns** with `group_nest()`; building paths with `str_glue()`
- Two inputs at once: `map2()` / `walk2()`; saving many CSVs and plots
- In-class activity

Helpful links

- Syllabus · AI policy · Schedule · *R4DS* Ch. 26 · purrr reference

The map() family

From across() to map()

Last class, `across()` repeated an action over the **columns** of a data frame.

`map()` is more general in that it applies a function to **every element of any list or vector** and returns the results. For example:

```
map(1:3, \(x) x^2)
```

```
[[1]]
[1] 1

[[2]]
[1] 4

[[3]]
[1] 9
```

In general, `map(x, f)` calls `f` once per element of `x` and collects the answers into a **list**. - Note that `\(x)` is the same anonymous-function shorthand we covered last class.

Typed variants return vectors

Because we don't always want a list returned, there are type-specific variants of `map()` as well. These typed variants return an atomic vector, and throw an error if the result isn't that type:

```
map_dbl(1:3, \(x) x^2)                    # double vector
```

```
[1] 1 4 9
```

```
map_chr(c("a", "bb"), \(x) str_c(x, "!"))   # character vector
```

```
[1] "a!"  "bb!"
```

```
map_int(list(1:5, 1:10), length)          # integer vector
```

```
[1]  5 10
```

Function	Returns
`map()`	a list
`map_dbl()` / `map_int()`	numeric / integer vector
`map_chr()`	character vector
`map_lgl()`	logical vector

A data frame is a list of columns

Because a data frame is really a list of columns, `map()` "walks" over its columns:

```
nums <- diamonds |> select(carat, depth, price)
map_dbl(nums, median)
```

```
 carat  depth  price
   0.7   61.8 2401.0
```

In essence, `map_dbl(df, median)` is another route that arrives at the same place as `summarize(across(everything(), median))`.

> **Note**
>
> You may see older code with `~ .x + 1` instead of `\(x) x + 1`. That tilde syntax only works inside tidyverse functions and always names the argument `.x`. The base `\(x)` form is preferrable.

map() reads many files

A classic use for `map()` is to read a folder of files and stack them together. First, you get the paths, `map()` `read_csv` over them, and then `list_rbind()` to bind the list of data frames into one:

```
paths <- list.files("data/gapminder", pattern = "\\.csv$", full.names = TRUE)

gapminder <- paths |>
  map(read_csv) |>    # a list of data frames, one per file
  list_rbind()        # stacked into a single data frame
```

In this class we tackle the **opposite** problem; that is, taking R objects and **saving them to many files**.

Saving many outputs

walk(): map() that throws the output away

When you call a function only for its **side-effect** (e.g., writing a file, printing, saving a plot), you don't necessarily care about the returned value. `walk()` does the same thing as `map()` without returning the output. Instead, it returns the input only:

```r
walk(c("first", "second", "third"), \(x) cat(x, "\n"))
```

```
first
second
third
```

Compare this to `map()`, which in addition to the inputs, also returns a list of `NULL`s.

```r
map(c("first", "second", "third"), \(x) cat(x, "\n"))
```

```
first
second
third

[[1]]
NULL

[[2]]
NULL

[[3]]
NULL
```

These NULLs aren't giving you any helpful information, and instead just clutter your console.

Example: many files into a database

When data won't all fit in memory, one alternative is to load it into a database. Then, you can query just the parts you need. The pattern is to create a **template** table, then **append** each file.

```r
con <- DBI::dbConnect(duckdb::duckdb())

# 1. build an empty table with the right columns/types from one file
template <- readxl::read_excel(paths[[1]])
template$year <- 1952
DBI::dbCreateTable(con, "gapminder", template)

# 2. a function that reads ONE file and appends it
```

```r
append_file <- function(path) {
  df <- readxl::read_excel(path)
  df$year <- parse_number(basename(path))
  DBI::dbAppendTable(con, "gapminder", df)
}

# 3. walk() it over every path (we don't need a return value)
paths |> walk(append_file)
```

It's better to use `walk()` here, instead of `map()`. This is because we only care about the rows landing in the database.

List-columns with group_nest()

To save one file per group, first split the data into per-group tibbles. `group_nest()` packs each group's rows into a **list-column** called `data`:

```r
by_clarity <- diamonds |> group_nest(clarity)
by_clarity
```

```
# A tibble: 8 x 2
  clarity               data
  <ord>   <list<tibble[,9]>>
1 I1                [741 x 9]
2 SI2             [9,194 x 9]
3 SI1            [13,065 x 9]
4 VS2            [12,258 x 9]
5 VS1             [8,171 x 9]
6 VVS2            [5,066 x 9]
7 VVS1            [3,655 x 9]
8 IF              [1,790 x 9]
```

Each element of `data` is a full tibble:

```r
by_clarity$data[[1]]
```

```
# A tibble: 741 x 9
   carat cut       color depth table price     x     y     z
   <dbl> <ord>     <ord> <dbl> <dbl> <int> <dbl> <dbl> <dbl>
```

```
 1  0.32 Premium   E      60.9    58   345  4.38  4.42  2.68
 2  1.17 Very Good J      60.2    61  2774  6.83  6.9   4.13
 3  1.01 Premium   F      61.8    60  2781  6.39  6.36  3.94
 4  1.01 Fair      E      64.5    58  2788  6.29  6.21  4.03
 5  0.96 Ideal     F      60.7    55  2801  6.37  6.41  3.88
 6  1.04 Premium   G      62.2    58  2801  6.46  6.41  4
 7  1    Fair      G      66.4    59  2808  6.16  6.09  4.07
 8  1.2  Fair      F      64.6    56  2809  6.73  6.66  4.33
 9  0.43 Very Good E      58.4    62   555  4.94  5     2.9
10  1.02 Premium   G      60.3    58  2815  6.55  6.5   3.94
# i 731 more rows
```

Build the output paths with str_glue()

If we want to add a column of file names, we can use `str_glue()` to interpolate the value of `clarity` into each string:

```
by_clarity <- by_clarity |>
  mutate(path = str_glue("diamonds-{clarity}.csv"))
by_clarity
```

```
# A tibble: 8 x 3
  clarity               data path
  <ord>   <list<tibble[,9]>> <glue>
1 I1                [741 x 9] diamonds-I1.csv
2 SI2             [9,194 x 9] diamonds-SI2.csv
3 SI1            [13,065 x 9] diamonds-SI1.csv
4 VS2            [12,258 x 9] diamonds-VS2.csv
5 VS1             [8,171 x 9] diamonds-VS1.csv
6 VVS2            [5,066 x 9] diamonds-VVS2.csv
7 VVS1            [3,655 x 9] diamonds-VVS1.csv
8 IF              [1,790 x 9] diamonds-IF.csv
```

We now have everything we need side by side, i.e., the **data** to write and the **path** to write it to.

Two inputs at once: map2() / walk2()

Saving needs **two** things that vary together: the data *and* the path. Writing this by hand:

```
write_csv(by_clarity$data[[1]], by_clarity$path[[1]])
write_csv(by_clarity$data[[2]], by_clarity$path[[2]])
# ... eight times (a lot of writing)
```

`map2()` varies the **first two** arguments together; `walk2()` does that and discards the output:

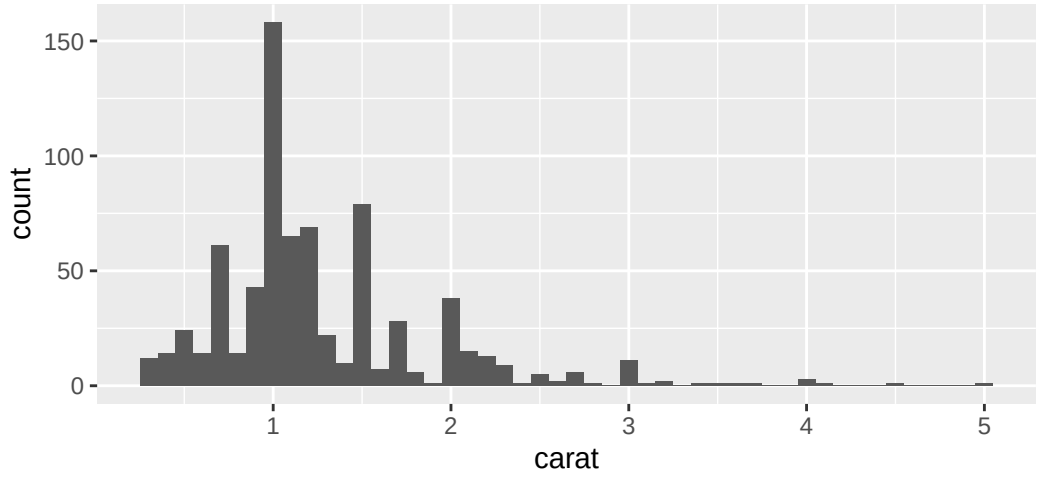
```
walk2(by_clarity$data, by_clarity$path, write_csv)
```

One line writes all eight CSVs. `walk2(x, y, f)` calls `f(x[[i]], y[[i]])` for each `i`.

Saving many plots

To save many plots, we first write a function that builds the plot for one data frame:

```
carat_histogram <- function(df) {
  ggplot(df, aes(x = carat)) + geom_histogram(binwidth = 0.1)
}
carat_histogram(by_clarity$data[[1]])
```



Saving many plots

Then, we can use `map()` to make a **list-column of plots**, and `str_glue()` for the paths:

```r
by_clarity <- by_clarity |>
  mutate(
    plot = map(data, carat_histogram),
    path = str_glue("clarity-{clarity}.png")
  )
```

Finally, `walk2()` + `ggsave()` writes them all:

```r
walk2(
  by_clarity$path,
  by_clarity$plot,
  \(path, plot) ggsave(path, plot, width = 6, height = 6)
)
```

So we can read files, write files, draw plots, and do it all with the same consistent `map`/`walk` pattern of code.

Why this matters

Iteration is powerful

The whole arc of this chapter is one idea applied three ways:

Task	Tool
do something to **each column**	`across()`
do something to **each element**	`map()` / `map_dbl()` / …
do something for its **side effect**	`walk()` / `walk2()`

> *"If you know the right iteration technique, you can easily go from fixing one problem to fixing all the problems." - R4DS*

What about for loops?

You may have noticed we haven't written a single `for` loop.

R's data-analysis orientation changes how we iterate: most of the time there's already an idiom. We do something to each **column** (`across`), each **group** (`group_by`), or each **element** (`map`).

`for` loops *do* exist. But a functional tool is almost always a better option to reach for first: it's shorter, harder to get wrong, and says *what* you mean, not just *how*.

Activity

In-class activity

Open `day22-activity.R` in RStudio and work through the parts in order.

- **Part 1**: `map()` and its typed variants; anonymous functions; `map()` over the columns of a data frame.
- **Part 2**: `group_nest()` to build a list-column, `str_glue()` for paths, and `walk2()` to write one CSV per group (to a temp folder).
- **Part 3 (challenge)**: write a plot function, build a list-column of plots with `map()`, and save them all with `walk2()` + `ggsave()`.

Activity file

Wrap-up & looking ahead

What we covered

- **`map(x, f)`** applies `f` to each element of `x` → a list
- Typed variants (`map_dbl`, `map_chr`, `map_int`, `map_lgl`) → vectors
- **`walk()`** = `map()` for side effects (writing, saving)
- **`group_nest()`** makes a per-group **list-column**
- **`str_glue()`** builds output paths from data
- **`map2()` / `walk2()`** vary **two** inputs together (data + path)
- Reading files, writing CSVs, saving plots

Upcoming

- **Thu Jun 18:** strings, dates & intro to regex + **Project oral check-in #1**
- **Reading before Thu:** *Data Computing* §17.1–17.2

Reach out

Stuck? Office hours, or email me (cgc5478@psu.edu) or Xinyue (xpw5228@psu.edu).

Reference

Syllabus · AI policy · Schedule · *R4DS* Ch. 26 · purrr reference